

Arithmetic Expression Trees (Part 1)

Cpt S 321 Homework Assignment

Washington State University

Read the entire HW before starting to work on it.

Overview of the steps to follow:

1. Create a branch called "Branch_HW5" and work in this branch for this assignment.
2. Read the Homework **Grading Rubric and the Points Breakdown** that appear at the end of this document.
3. Work on the Assignment Instructions below.
4. Create a pdf document with your AI statement for the HW (call it "AI_HW5"), stating whether or not you used GenAI and how and upload it in the "AI_Statements" folder. If you used GenAI for this HW, 1) detail the process you followed in this document and 2) follow the guidelines for committing code generated by AI and adapted by you as outlined in the syllabus in the AI Statement section.
5. When you are done, merge the branch back to the master.
6. Once you are done and before the deadline, tag the version that you would want us to grade with the assignment number. For example, "HW5".
7. **Record a video to demo your HW (call it "Demo_HW5-FirstName_LastName").**
8. On Canvas -> Assignments -> Assignments -> **Upload your pre-recorded video by the HW deadline.**
9. Sign up for a live demo with the TAs before the deadline.

Assignment Instructions:

In this assignment you will create the **ExpressionTree** class in your logic engine DLL. You will create a standalone console application as well, to demo its functionality. The ExpressionTree class will implement an arithmetic expression parser that builds a tree for the expression. This tree can then be used for evaluation of the expression. The parsing aspect of this assignment is simplified and you will extend the parser in the next homework. But the entire evaluation functionality, including setting variable values, will be implemented in this assignment.

Create a class named **ExpressionTree**. Create this class in your logic engine DLL. The console app for this assignment just references your engine DLL and provides a simple menu for a demo. The console application, the logic engine DLL, and your test project will be graded. Include the following methods in the ExpressionTree class:

- public ExpressionTree(string expression)

- Implement this constructor to construct the tree from the specific expression
- `public void SetVariable(string variableName, double variableValue)`
 - Sets the specified variable within the ExpressionTree variables dictionary
- `public double Evaluate()`
 - Implement this method with no parameters that evaluates the expression to a double value

Create a console application that presents a menu when run. This menu must contain the following options:

1. The option to enter an expression string. *You may assume that only valid expressions will be entered with no whitespaces.* Simplified expressions are used for this assignment and the assumptions you are allowed to make are discussed later on.
2. The option to set a variable value in the expression. This must prompt for both the variable name and then the variable value.
3. The option to evaluate the expression to a numerical value.
4. The option to quit.
5. Should the user enter an “option” that isn’t one of these 4, simply ignore it. As trivial as this may seem it is vital should the assignment be tested with an automated grading app.
 - On that note, also avoid use of `Console.ReadKey()` as it can be problematic when grading with an automated app. Simply fall through to the end of main after the quit option is selected.

The screenshot below shows what this might look like:

```
Debug — bash — 118x43
Menu (current expression="A1-12-C1")
1 = Enter a new expression
2 = Set a variable value
3 = Evaluate tree
4 = Quit
1
Enter new expression: Hello-12-World
Menu (current expression="Hello-12-World")
1 = Enter a new expression
2 = Set a variable value
3 = Evaluate tree
4 = Quit
2
Enter variable name: Hello
Enter variable value: 42
Menu (current expression="Hello-12-World")
1 = Enter a new expression
2 = Set a variable value
3 = Evaluate tree
4 = Quit
2
Enter variable name: World
Enter variable value: 20
Menu (current expression="Hello-12-World")
1 = Enter a new expression
2 = Set a variable value
3 = Evaluate tree
4 = Quit
// This is a command that the app will ignore
Menu (current expression="Hello-12-World")
1 = Enter a new expression
2 = Set a variable value
3 = Evaluate tree
4 = Quit
3
10
Menu (current expression="Hello-12-World")
1 = Enter a new expression
2 = Set a variable value
3 = Evaluate tree
4 = Quit
4
Done
```

Requirement Details:

Support simplified expressions:

- For this assignment all expressions will be simplified next to what you'll have to support in the next homework. As stated earlier, this assignment is primarily about getting the correct evaluation logic in place for the tree.
- Assume expressions will NOT have any parentheses
- Assume expressions will only have a single *type* of operator, but can have any number of instances of that operator in the expression
 - Example 1: expression could be "A+B+C1+Hello+6"
 - Example 2: expression could be "C2-9-B2-27"
 - Example 3: expression could NOT be "X+Y-Z" because that has two different types of operators: + and -
- (In the next assignment you'll have to support all valid arithmetic expressions)
- Support operators +, -, *, and / for addition, subtraction, multiplication, and division, respectively. Again, only one type will be present in an expression that the user enters.
- Parse the expression that the user enters and build the appropriate tree in memory

Tree Construction and Evaluation:

- Build the expression tree correctly internally. Non-expression-tree-based implementations will not be worth any points.
- Each node in the tree will be in one of three categories:
 - Node representing a constant numerical value
 - Node representing a variable
 - Node representing a binary operator
- The above three types should match nicely with a design of a base class for the node and then 3 inheriting classes.
 - No node should store any more data than it needs to. As just one example of what this means, the constant value and variable nodes never have children so their class declarations shouldn't have references to children (nor should they be inheriting such declarations from a parent class).
- Your ExpressionTree class must have a public evaluation function that takes no parameters and returns the value of the expression as a double.
 - `public double Evaluate()`

Support for Variables:

- Support correct functionality of variables including multi-character values (like "A2").
- Variables will start with an alphabet character, upper or lower-case, and be followed by any number of alphabet characters and numerical digits (0-9).
- A set of variables is stored per-expression, so creating a new expression will clear out the old set of variables.
- Have a default expression, something like "A1+B1+C1" would be fine, so that if setting variables is the first action that the user chooses then you have an expression object to work with.
- If variables are not set by the user, they can be default to 0 for this HW. In later HWs, once we learn how to deal with exceptions, we will change this.

Homework Grading Rubric and Points Breakdown	
CR1: Functionality (4 points)	● Fulfill all the requirements above with no inaccuracies in the output and no crashes.
CR2 Quality of Version Control (1 point)	● Homework should be built iteratively (i.e., one feature at a time, not in one huge commit). ● Each commit should have cohesive functionality. ● Commit messages should concisely describe what is being committed. ● TDD should be used (i.e, write and commit tests first and then implement and commit functionality).

	● Include “TDD” in all commit messages with tests that are written before the functionality is implemented. ● Use of a .gitignore. ● Commenting is done as the homework is built (i.e, there is commenting added in each commit, not done all at once at the end).
CR3: Quality of Identifiers (1 point)	● No underscores in names of classes, attributes, and properties. ● No numbers in names of classes or tests. ● Identifiers should be descriptive. ● Project names should make sense. ● Class names and method names use PascalCasing. ● Method arguments and local variables use camelCasing. ● No Linguistic Antipatterns or Lexicon Bad Smells.
CR4: Existence and Quality of Comments (1 point)	● Every method, attribute, type, and test case has a leading comment block with a minimum of <summary>, <returns>, <param>, and <exception> filled in as applicable. ● All comment blocks use the format that is generated when typing “///” on the line above each entity. ● There is useful inline commenting <u>in addition to leading comment blocks</u> that explains how the algorithm is implemented as needed.
CR5: Quality of Code (1 point)	● Each file should only contain one public class. ● Correct use of access modifiers. ● Classes are cohesive. ● Namespaces make sense. ● Code is easy to follow. ● StyleCop is installed and configured correctly for all projects in the solution and all warnings are resolved. If any warnings are suppressed, a good reason must be provided. ● Use of appropriate design patterns and software principles seen in class.
CR6: Existence and Quality of Tests (1 point)	● Normal, boundary, and overflow/error cases should be tested – tests should be written with NUnit. ● Test should be modularized (i.e, have a separate test case for each functionality - do not combine them into one large test case). ● <i>Note: In assignments with a GUI, we do not require testing of the GUI itself.</i>
CR7: Demo video – 5 min max (1 point)	● Check the “Pre-recorded demos instructions” in the additional resources for details on the items below.

	• Version control history overview (< 30 sec.) • AI statement (< 30 sec.) • Design of the solution (2 min max) • Features showcase (2 min max)
--	--